

lab05_256429_FabianoJunior

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1      ✓ readr      2.2.0
✓ forcats    1.0.1      ✓ stringr    1.6.0
✓ ggplot2    4.0.3      ✓ tibble     3.3.1
✓ lubridate  1.9.5      ✓ tidyr      1.3.2
✓ purrr      1.2.2

— Conflicts — tidyverse_conflicts()
—
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
library(readr)
library(ggplot2)
library(leaflet)

# Ler o arquivo dos aeroportos
airports <- read_csv(
  "airports.csv",
  col_select = c(
    IATA_CODE,
    CITY,
    STATE,
    LATITUDE,
    LONGITUDE
  ),
  show_col_types = FALSE
)

# Função para analisar uma aeronave
analisa_aeronave <- function(tail_number, arquivo) {
```

```

# Lista para armazenar os resultados de cada parte do arquivo
partes <- list()

# Função executada para cada parte do flights.csv
callback <- DataFrameCallback$new(
  function(x, pos) {

    # Selecionar somente os voos da aeronave
    dados <- x %>%
      filter(
        TAIL_NUMBER == tail_number,
        CANCELLED == 0
      )

    # Armazenar os voos encontrados
    if (nrow(dados) > 0) {
      partes[[length(partes) + 1]] <- dados
    }
  }
)

# Ler flights.csv por partes de 1 milhão de linhas
read_csv_chunked(
  arquivo,
  callback = callback,
  chunk_size = 1000000,
  progress = FALSE
)

# Juntar todas as partes
voos <- bind_rows(partes)

# Criar data e ordenar os voos
voos <- voos %>%
  mutate(
    DATA = as.Date(
      paste(YEAR, MONTH, DAY, sep = "-")
    ),
    HORA = as.numeric(SCHEDULED_DEPARTURE)
  )

```

```

) %>%
arrange(DATA, HORA)

# -----
# TABELA
# -----

tabela <- voos %>%
  select(
    all_of(names(read_csv(
      arquivo,
      n_max = 0,
      show_col_types = FALSE
    )))
  )

# -----
# DADOS PARA O GRÁFICO
# -----

dados_grafico <- voos %>%
  mutate(
    velocidade_media = DISTANCE / AIR_TIME * 60
  ) %>%
  filter(
    is.finite(velocidade_media)
  )

# Gráfico da velocidade média dos voos
grafico <- ggplot(
  dados_grafico,
  aes(
    x = DATA,
    y = velocidade_media
  )
) +
  geom_point() +
  labs(
    title = paste(
      "Velocidade média dos voos da aeronave",

```

```

        tail_number
      ),
      x = "Data",
      y = "Velocidade média"
    ) +
    theme_minimal()

# Retornar tabela e gráfico
return(
  list(
    tabela = tabela,
    grafico = grafico
  )
)
}

# -----
# ANALISAR A AERONAVE
# -----

resultado <- analisa_aeronave(
  "N431WN",
  "flights.csv"
)

```

```

—
specification _____ Column
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),

```

```

    CANCELLATION_REASON = col_character()
  )
  i Use `spec()` for the full column specifications.

```

```

# Mostrar a tabela
resultado$tabela

```

```

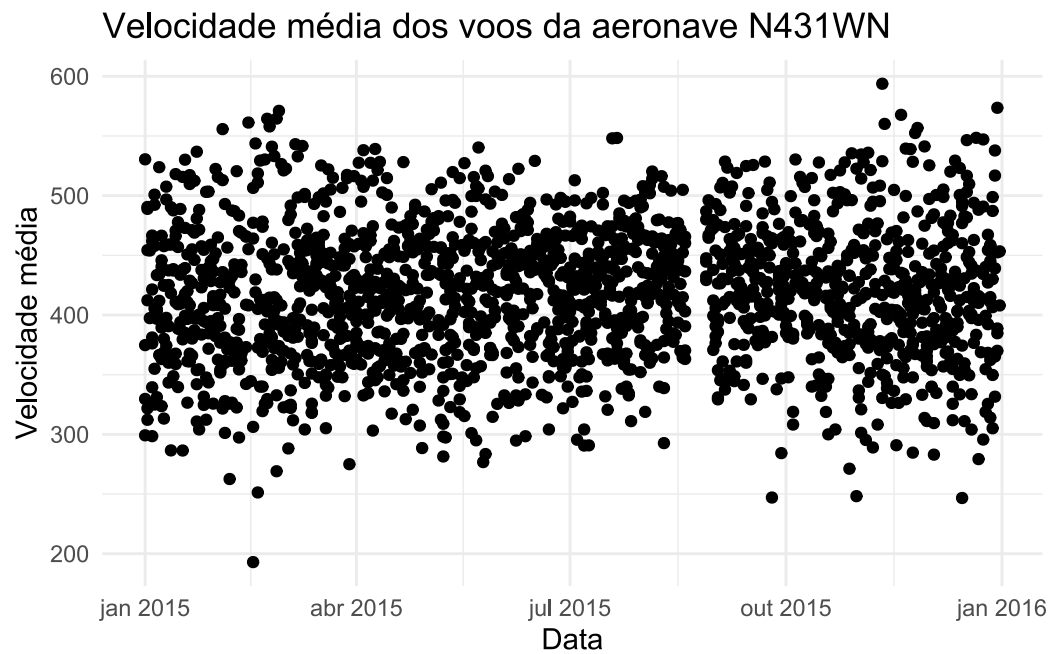
# A tibble: 1,926 × 31
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER
  <dbl> <dbl> <dbl>      <dbl> <chr>      <dbl> <chr>
1  2015     1     1          4 WN          1762 N431WN
2  2015     1     1          4 WN          3564 N431WN
3  2015     1     1          4 WN          3564 N431WN
4  2015     1     1          4 WN          3564 N431WN
5  2015     1     2          5 WN          1793 N431WN
6  2015     1     2          5 WN          1793 N431WN
7  2015     1     2          5 WN          1793 N431WN
8  2015     1     2          5 WN          1793 N431WN
9  2015     1     2          5 WN          1793 N431WN
10 2015     1     2          5 WN          3144 N431WN
# i 1,916 more rows
# i 24 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
#   SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,
#   TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
#   AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
#   SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
#   DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...

```

```

# Mostrar o gráfico
resultado$grafico

```



```
# -----  
# MAPA  
# -----  
  
mapa_dados <- resultado$tabela %>%  
  left_join(  
    airports %>%  
      select(  
        IATA_CODE,  
        LATITUDE,  
        LONGITUDE  
      ),  
    by = c(  
      "ORIGIN_AIRPORT" = "IATA_CODE"  
    )  
  ) %>%  
  rename(  
    ORIGIN_LAT = LATITUDE,  
    ORIGIN_LON = LONGITUDE  
  ) %>%  
  left_join(  
    airports %>%  
      select(  
        IATA_CODE,
```

```

        LATITUDE,
        LONGITUDE
    ),
    by = c(
        "DESTINATION_AIRPORT" = "IATA_CODE"
    )
) %>%
rename(
    DESTINATION_LAT = LATITUDE,
    DESTINATION_LON = LONGITUDE
)

```

```

# Criar o mapa
mapa <- leaflet() %>%
  addTiles()

# Adicionar os trajetos
for (i in 1:nrow(mapa_dados)) {

  # Velocidade média do voo
  velocidade <- mapa_dados$DISTANCE[i] /
    mapa_dados$AIR_TIME[i] * 60

  # Espessura da linha baseada na velocidade
  espessura <- 1 + min(
    velocidade / 100,
    8
  )

  mapa <- mapa %>%
    addPolylines(
      lng = c(
        mapa_dados$ORIGIN_LON[i],
        mapa_dados$DESTINATION_LON[i]
      ),
      lat = c(
        mapa_dados$ORIGIN_LAT[i],
        mapa_dados$DESTINATION_LAT[i]
      ),
      weight = espessura,
      opacity = 0.7,

```

```
popup = paste0(  
  "Data: ", mapa_dados$DATA[i],  
  "<br>Voo: ", mapa_dados$FLIGHT_NUMBER[i],  
  "<br>Origem: ", mapa_dados$ORIGIN_AIRPORT[i],  
  "<br>Destino: ", mapa_dados$DESTINATION_AIRPORT[i],  
  "<br>Velocidade média: ",  
  round(velocidade, 2)  
)  
)  
}
```

mapa